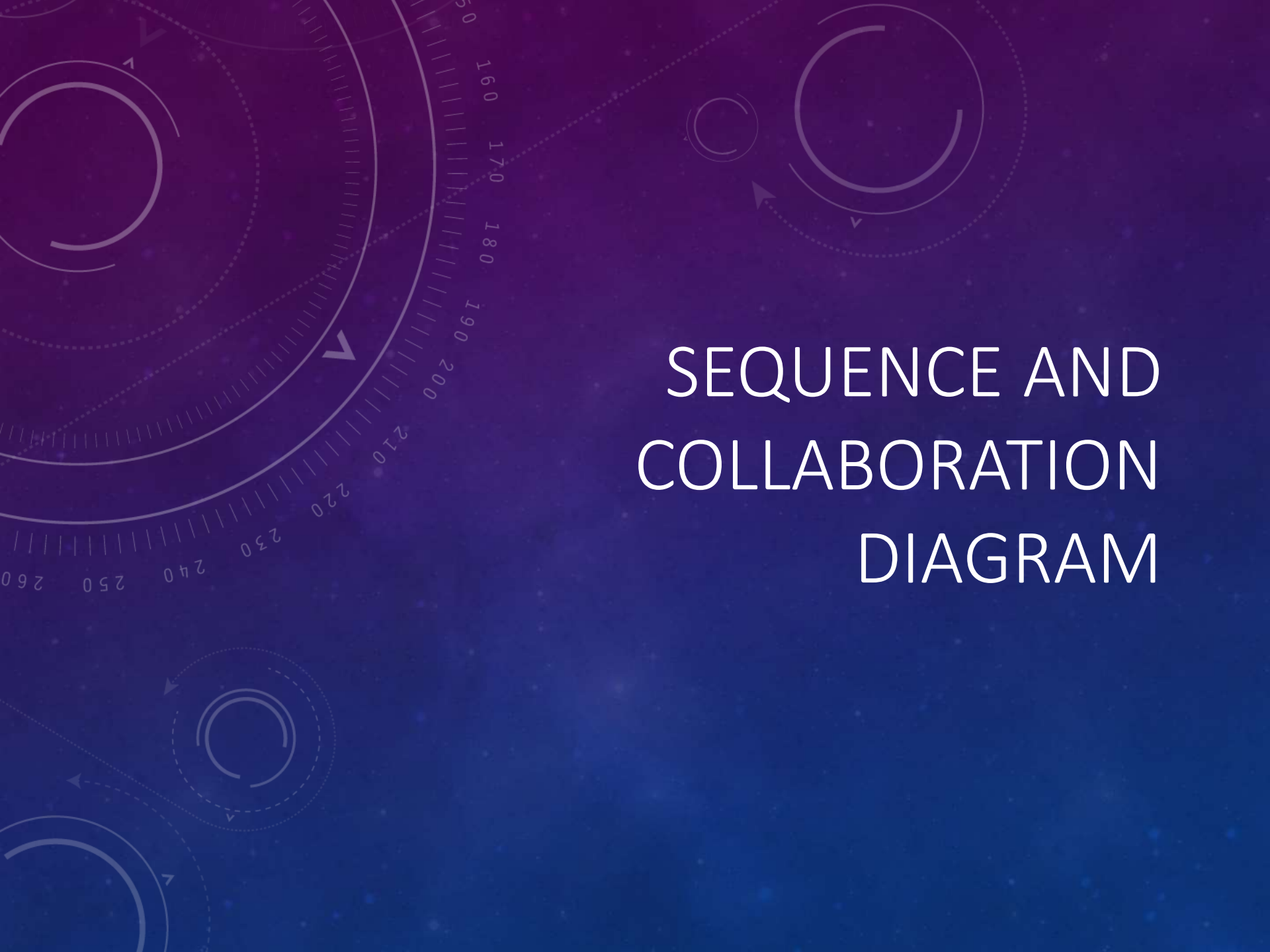
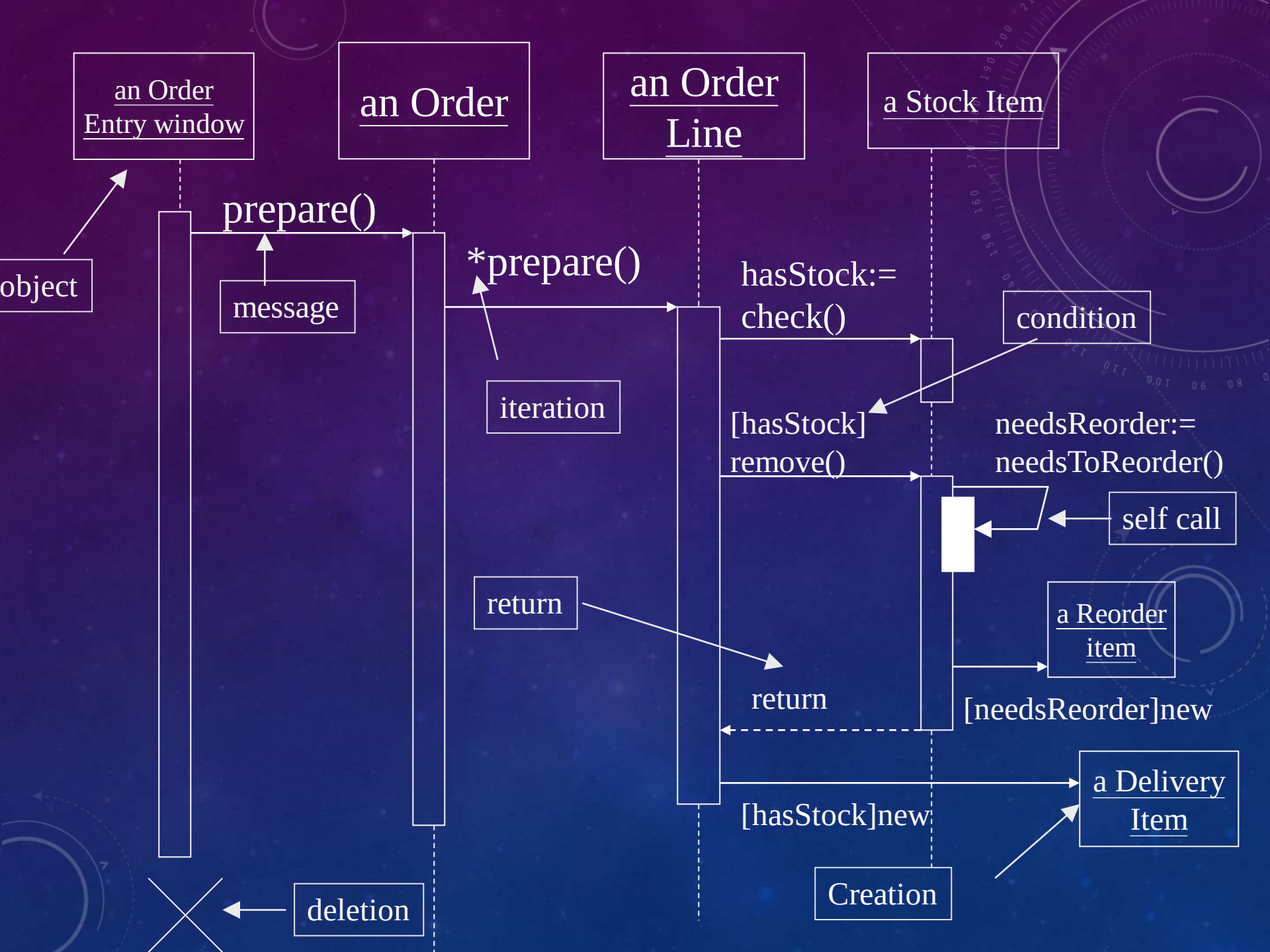


The background is a deep blue gradient with a subtle pattern of white dots. Overlaid on the left side are several faint, white circular elements. A large circular scale with tick marks and numbers (150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, 260) is prominent. Other circular patterns include concentric circles with arrows indicating clockwise or counter-clockwise movement, and dashed lines forming circular paths.

SEQUENCE AND COLLABORATION DIAGRAM

INTERACTION DIAGRAM

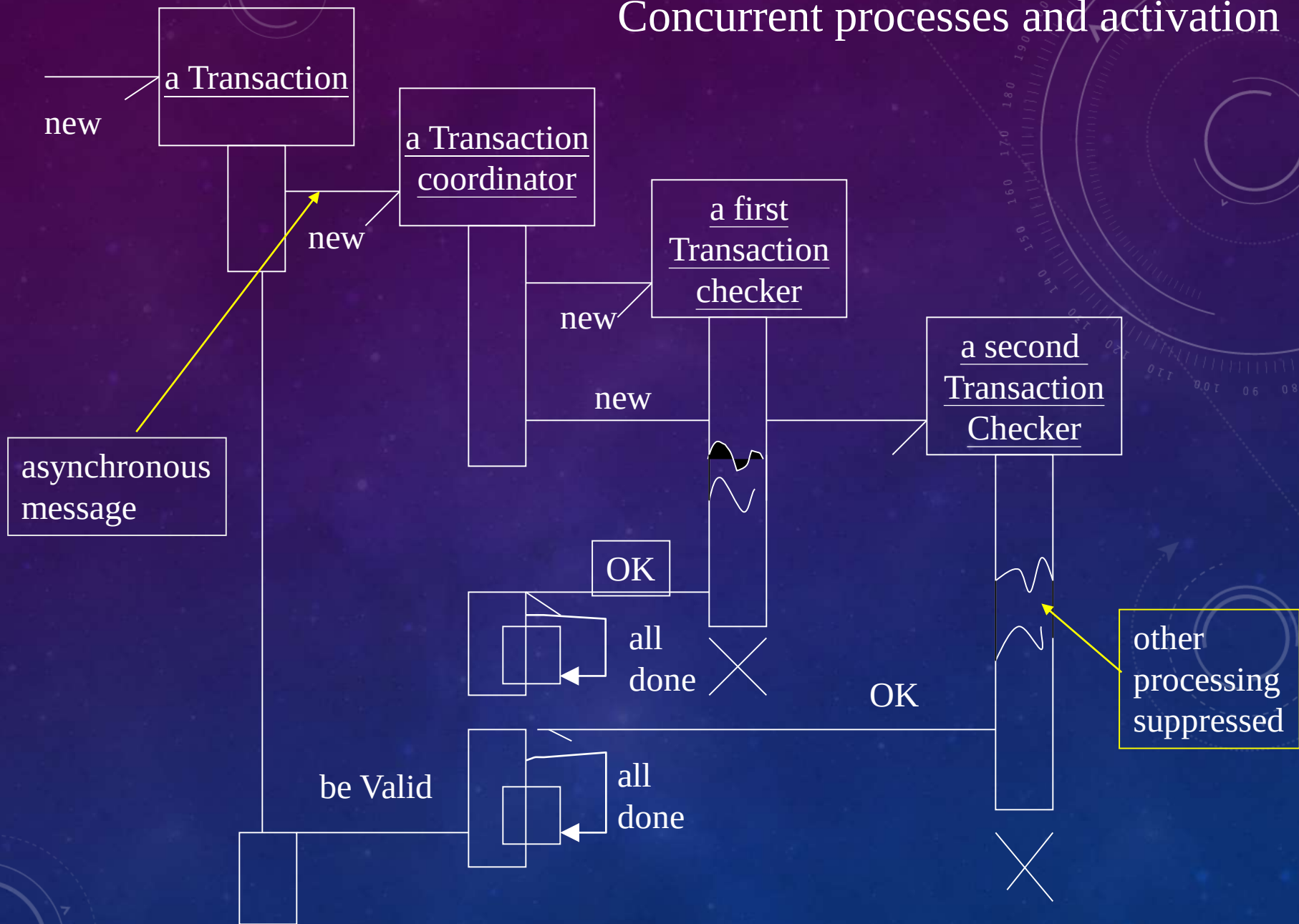
- models describe how groups of objects collaborate in some behavior
- NOTE THAT: not class
- typically, captures the behavior of a single use case
- Two kinds
 - sequence diagrams
 - collaboration diagrams



WHEN TO USE SEQUENCE DIAGRAM

- One of the hardest things to understand in an object-oriented program is the overall flow of control.
- A good design has lots of small methods in different classes and at times it can be tricky to figure out the overall sequence of behavior
- Sequence diagram helps you see the sequence
- valuable for concurrent process

Concurrent processes and activation



SEQUENCE DIAGRAM

- 請看一下下面兩張sequence diagram

Figure 4.1. A sequence diagram for centralized control

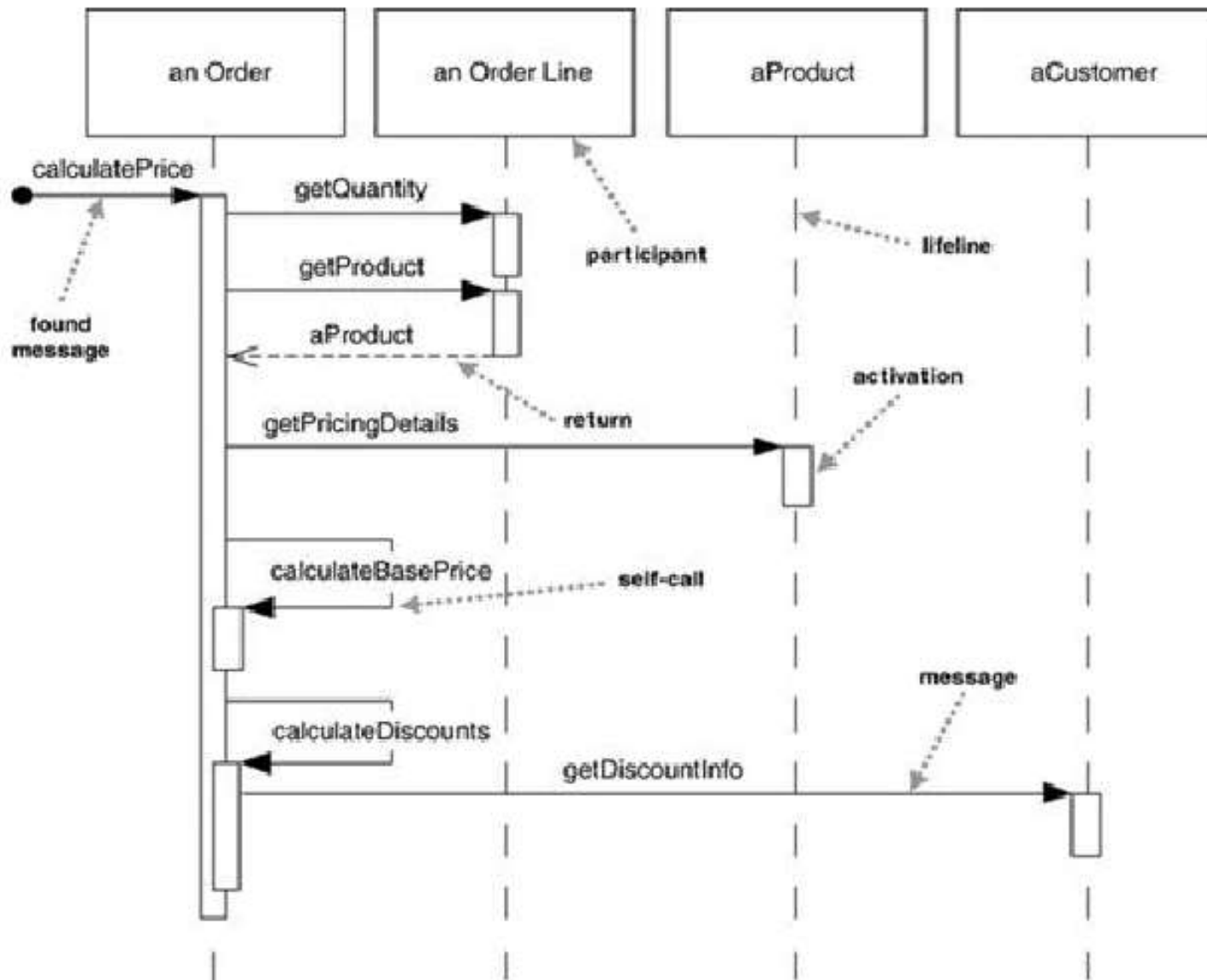
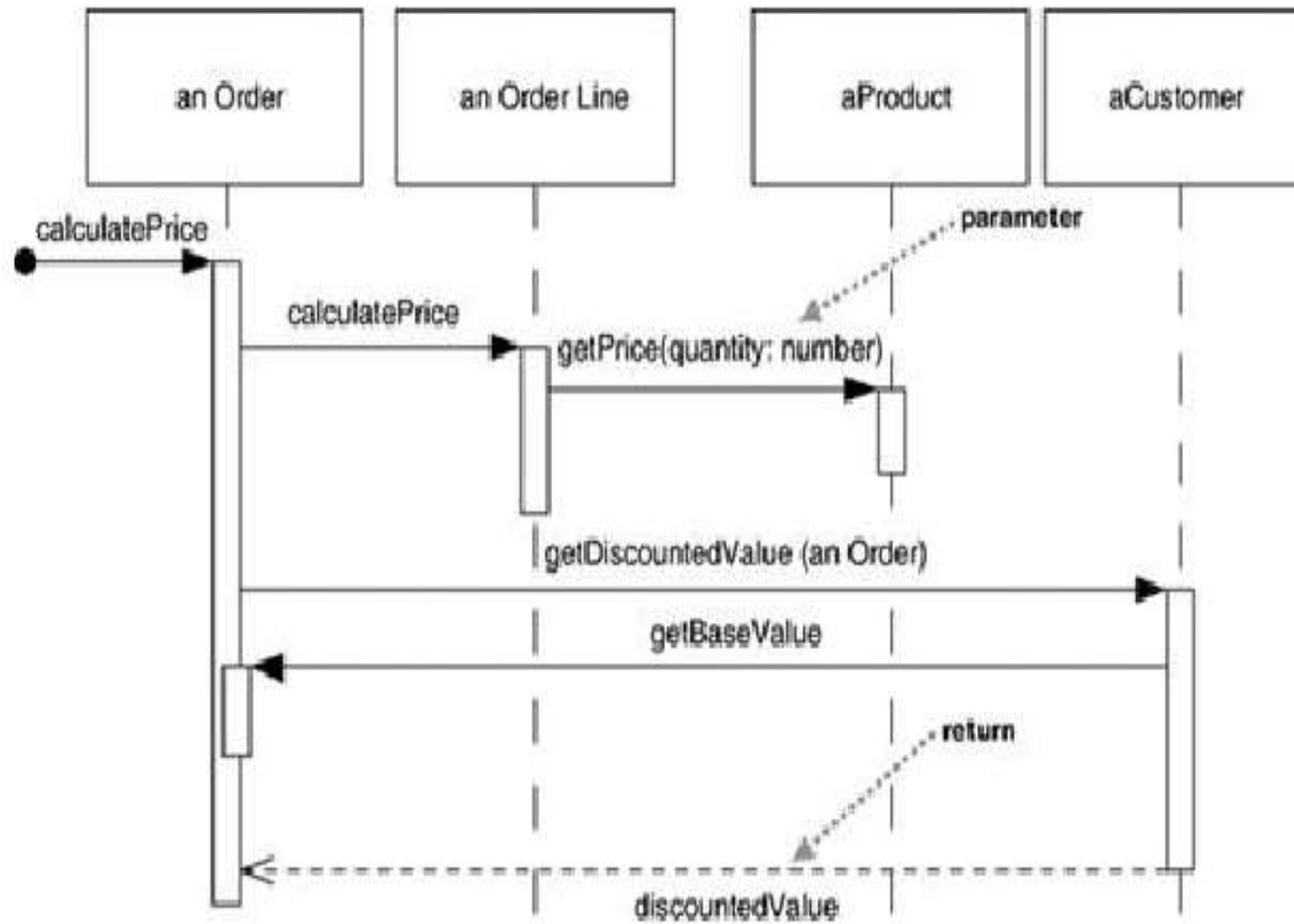


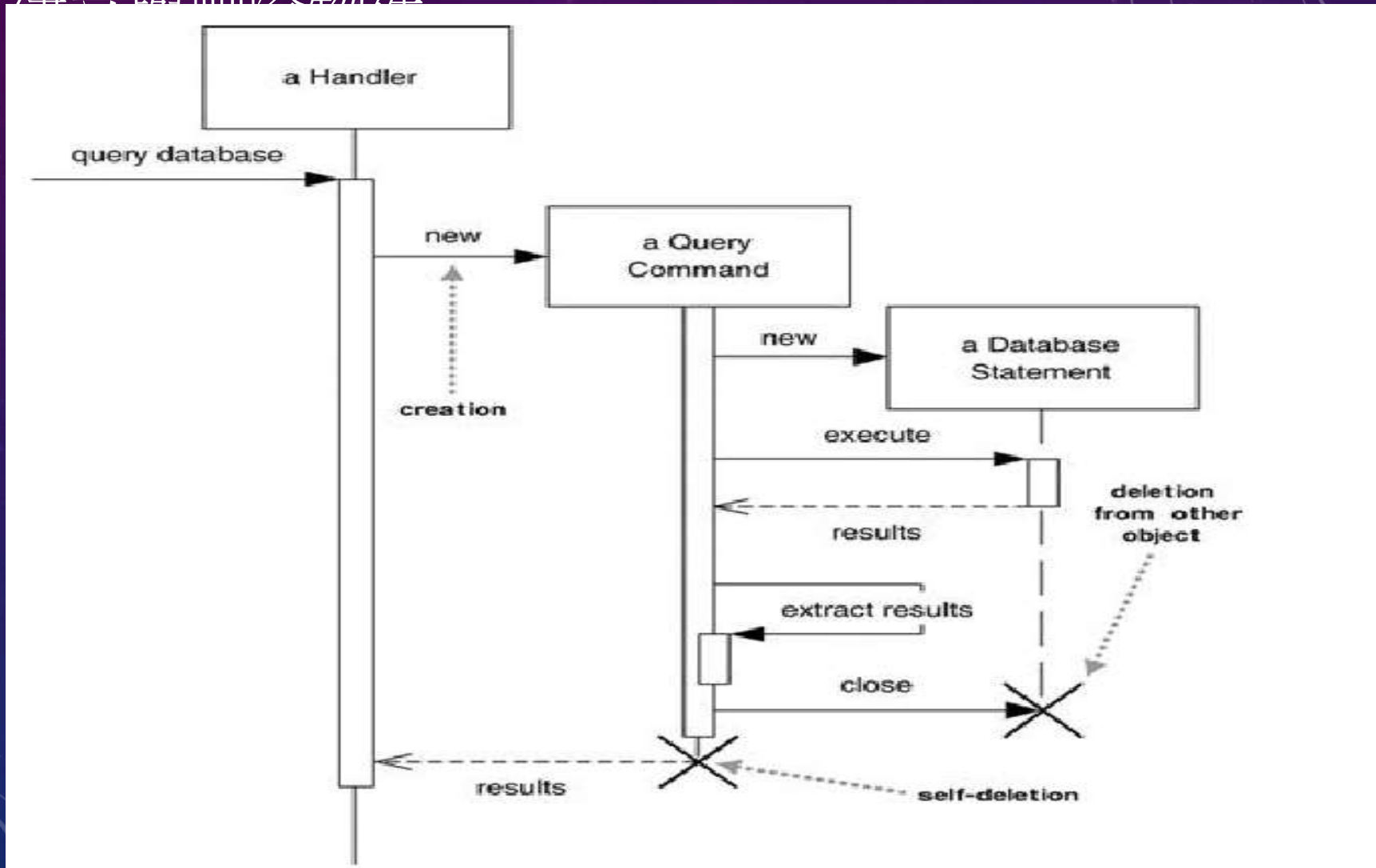
Figure 4.2. A sequence diagram for distributed control



COMMENTS

- Sequence diagram 並不擅長展示有 loop 等的演算法
- 適合展示某個複雜的互動情境

7.4.6 数据库命令

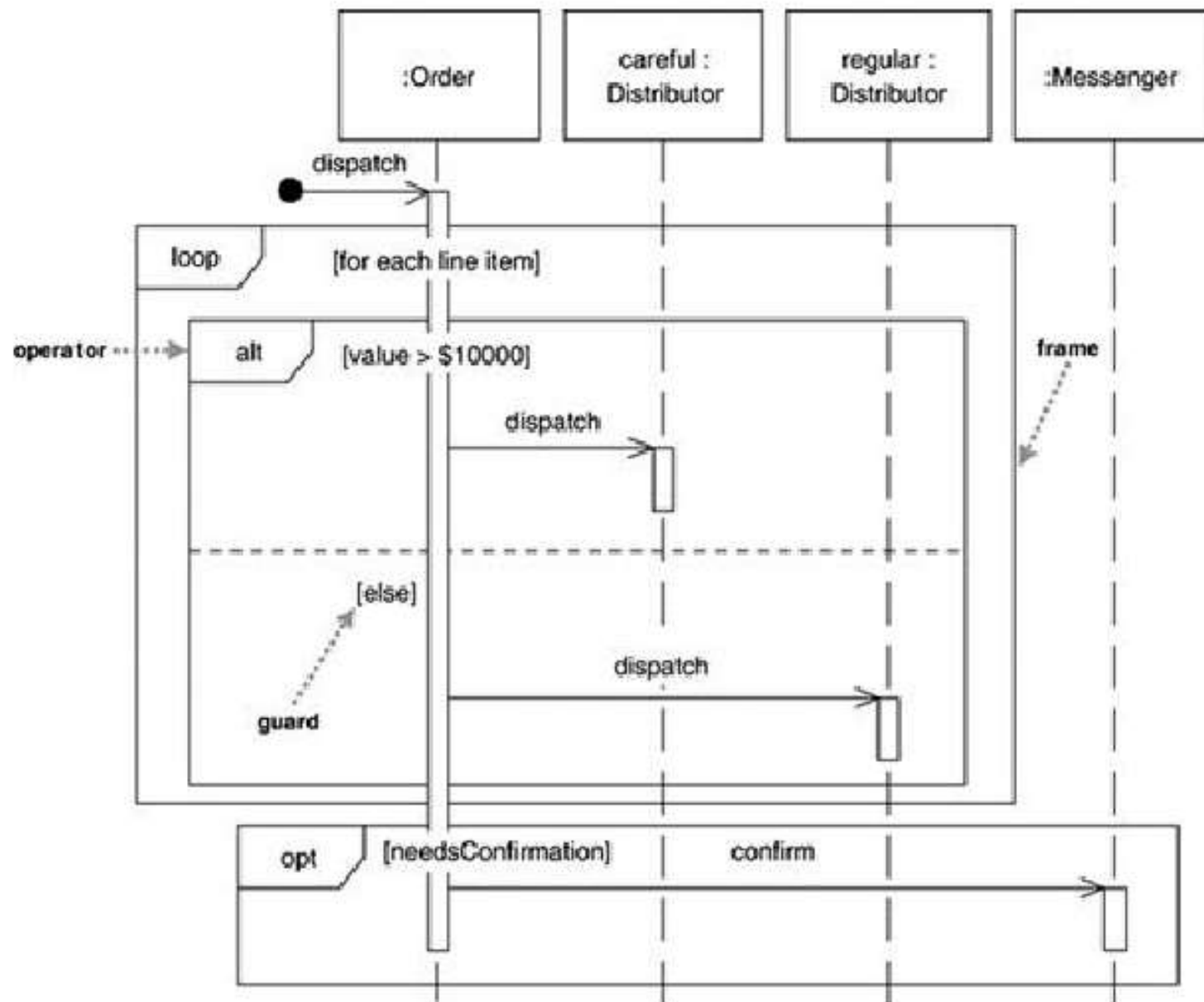


COMMENTS

- Sequence diagram 並不擅長展示有 loop 等的演算法

1. 適合展示某個複雜的互動情境 **Loops, Conditionals, and the Like**

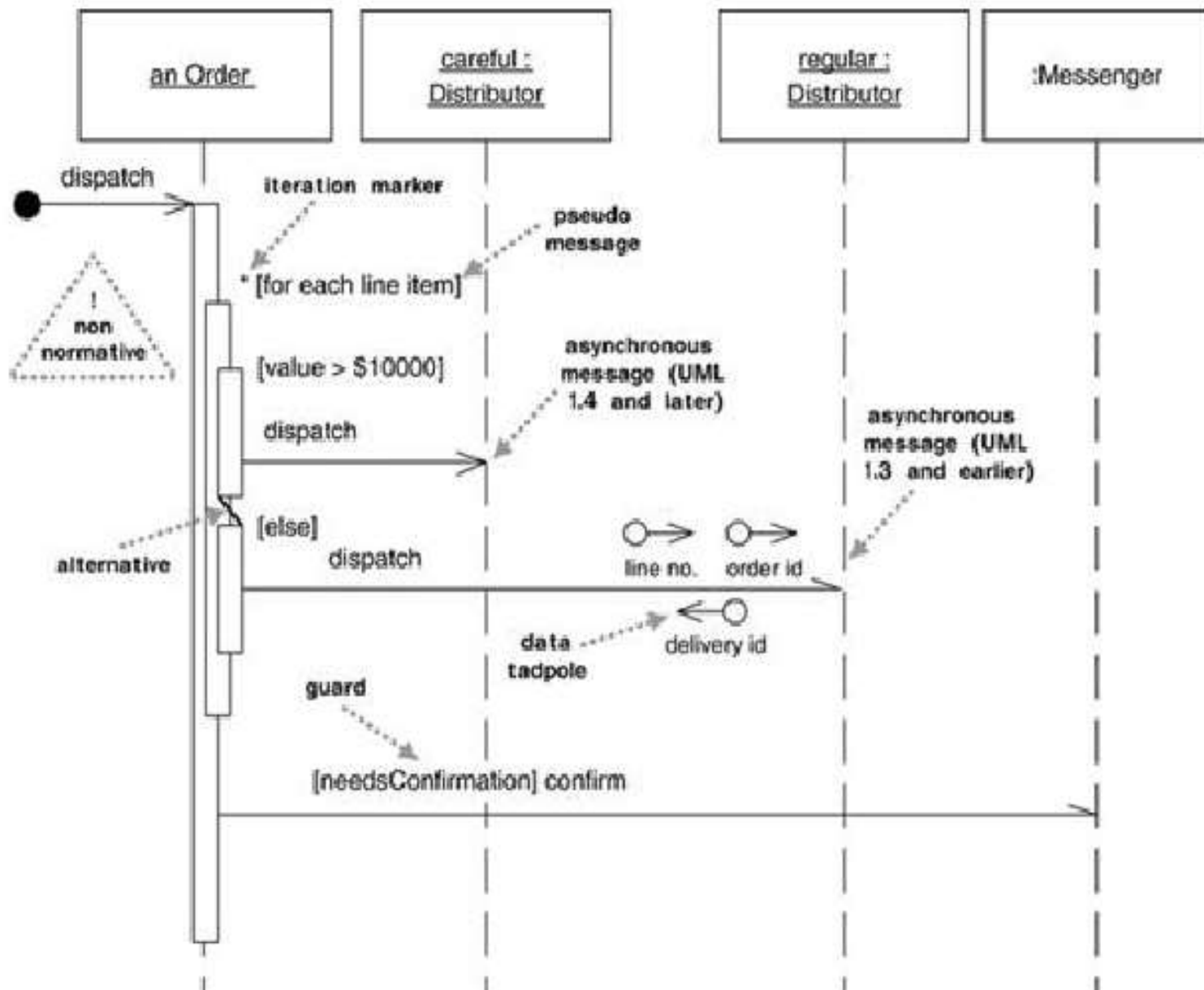
A common issue with sequence diagrams is how to show looping and conditional behavior. The first thing to point out is that this isn't what sequence diagrams are good at. If you want to show control structures like this, you are better off with an activity diagram or indeed with code itself. Treat sequence diagrams as a visualization of how objects interact rather than as a way of modeling control logic.



```
procedure dispatch
  foreach (lineitem)
    if (product.value > $10K)
      careful.dispatch
    else
      regular.dispatch
    end if
  end for
  if (needsConfirmation) messenger.confirm
end procedure
```

LOOP AND ALTERNATIVE FRAME

- Loop 以及 alternative 框架是 UML 2.0 之後才有的
- 有些人不喜歡 loop 以及 alternative 框架，而自行採用其他的方法如下圖

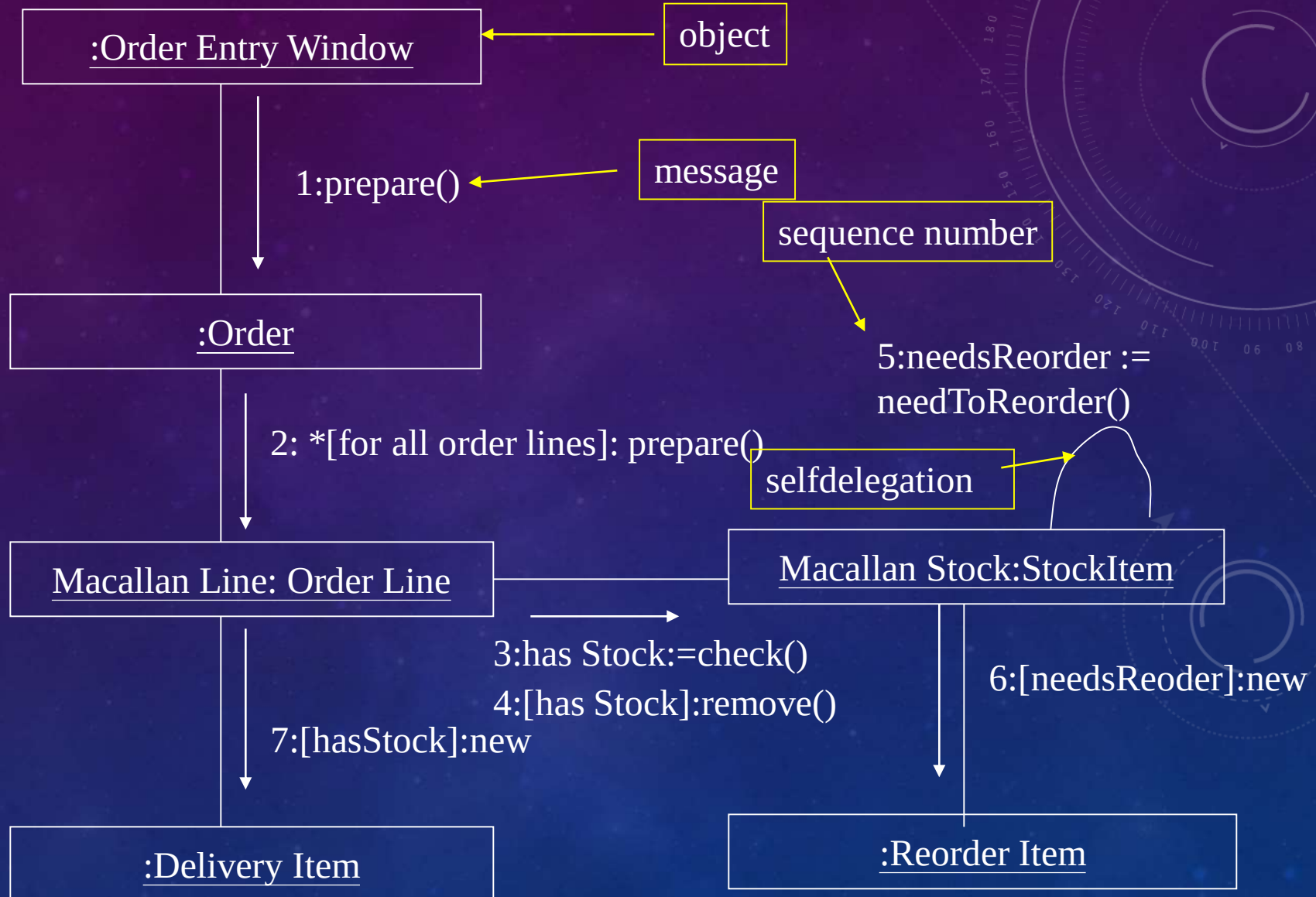


FRAME OPTION

- **alt** Alternative multiple fragments; only the one whose condition is true will execute (Figure 4.4).
- **opt** Optional; the fragment executes only if the supplied condition is true. Equivalent to an alt with only one trace (Figure 4.4).
- **par** Parallel; each fragment is run in parallel.
- **loop** Loop; the fragment may execute multiple times, and the guard indicates the basis of iteration (Figure 4.4).
- **region** Critical region; the fragment can have only one thread executing it at once.
- **neg** Negative; the fragment shows an invalid interaction.
- **ref** Reference; refers to an interaction defined on another diagram. The frame is drawn to cover the lifelines involved in the interaction. You can define parameters and a return value.
- **sd** Sequence diagram; used to surround an entire sequence diagram, if you wish.

COLLABORATION DIAGRAM

- The sequence is indicated by numbering the message
- the spatial layout allows you to show other things more easily
- show how object link together
- you can overlay packages or other information more easily



SEQUENCE OR COLLABORATION?

- different people have different preference for different situation
- interaction diagrams can awkward to use when exploring alternatives

HOW TO FIND METHODS (BEHAVIORS, OPERATIONS) OF OBJECT

- 好，你應該知道要怎麼樣以sequence diagram描述物件之間的互動
- 重點是，你的物件的 **method** 根本還沒有個譜？
- 你怎麼找他們？

9.2. THE KRB SEVEN-STEP METHOD.

STEP ⑦ OPERATIONS (I.E., BEHAVIOR)

Six techniques for Finding Operations

1. By Inspection
2. Basic *CRUD*
3. Use Cases
4. Statechart Diagram
5. CRC Cards
6. *CRUD* Revisited

WHAT IS A USE CASE ?

- 使用案例（系統使用情境 **scenario**）
- **Use cases** 是你的系統所要完成的功能
- 這些功能當然要由你所設計的 **objects** 互動來完成
- 請看上次的規格書

Figure 9.1. Example use case text

Buy a Product

Goal Level: Sea Level

Main Success Scenario:

1. Customer browses catalog and selects items to buy
2. Customer goes to check out
3. Customer fills in shipping information (address; next-day or 3-day delivery)
4. System presents full pricing information, including shipping
5. Customer fills in credit card information
6. System authorizes purchase
7. System confirms sale immediately
8. System sends confirming e-mail to customer

Extensions:

3a: Customer is regular customer

- .1: System displays current shipping, pricing, and billing information
- .2: Customer may accept or override these defaults, returns to MSS at step 6

6a: System fails to authorize credit purchase

- .1: Customer may reenter credit card information or may cancel

WHAT IS A SCENARIO (情境)

A **scenario** is a sequence of steps describing an interaction between a user and a system.

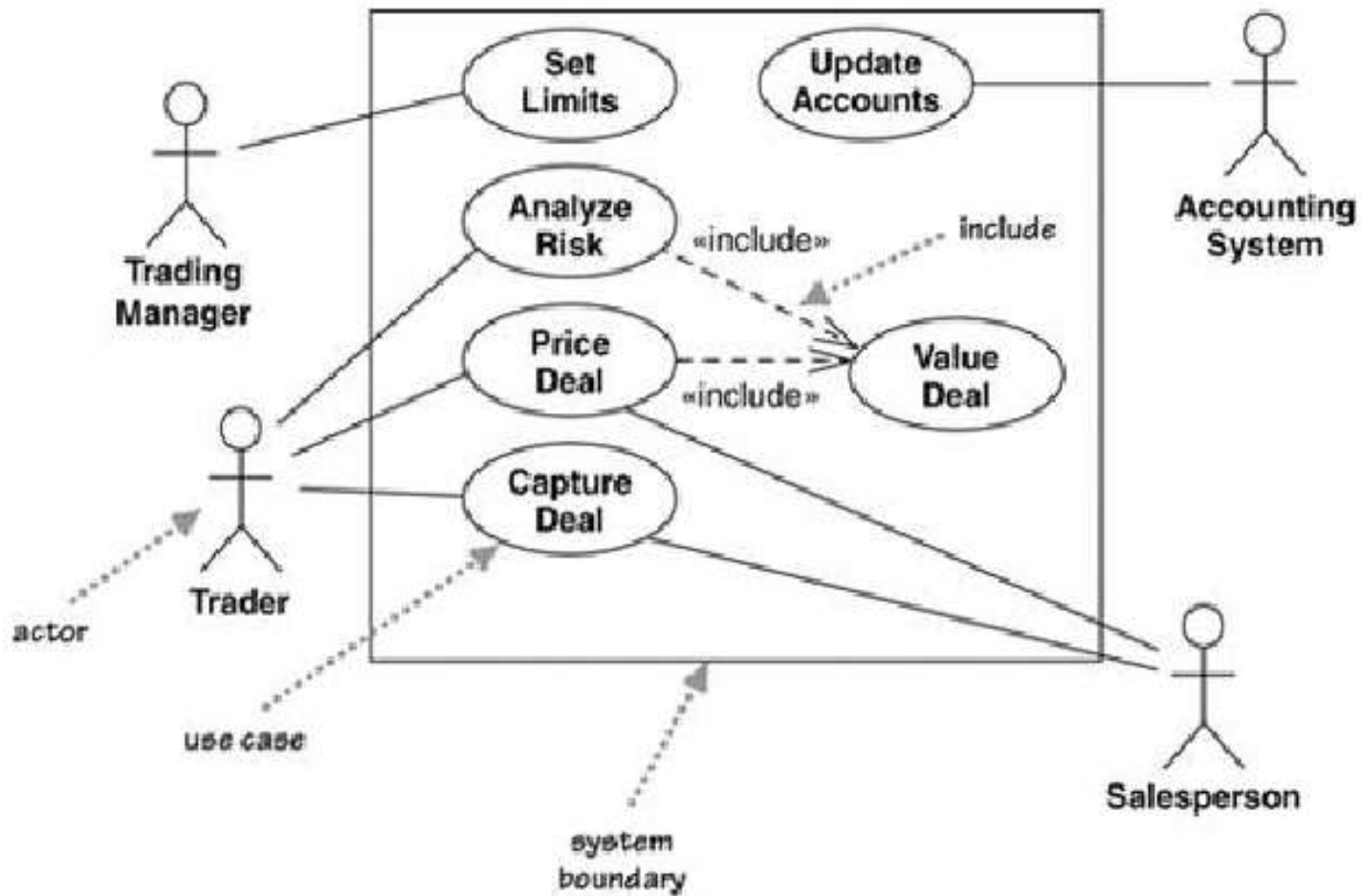
So if we have a Web-based on-line store, we might have a Buy a Product scenario that would say this:

The customer browses the catalog and adds desired items to the shopping basket. When the customer wishes to pay, the customer describes the shipping and credit card information and confirms the sale. The system checks the authorization on the credit card and confirms the sale both immediately and with a follow-up e-mail.

WHAT IS A USE CASE?

- 一個使用案例是一組（系統使用，或系統操作）情境來達成某個目標
- 一個趨勢是規格書的寫法漸漸由 **use cases** 所取代，尤其是軟體代工產業

USE CASE DIAGRAM



USE CASE干尋找物件METHOD啥事

- 一旦有了 use case (可以有正式文件，或者是 建模者自行寫下一些scenario)
- Scenarios (use cases) 是系統行為的起頭
- 每一個use case 最終一定要由系統中的物件來完成
- OK，接著？